VL 04 · JavaScript kompakt

Die Hackathon-Seite wird lebendig: Sprache, DOM, Events, Event Loop und Fetch

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27

Unsere Hackathon-Seite sieht gut aus, kann aber nichts

Programm fest im HTML

Ändert sich ein Termin, muss jemand das HTML anfassen. Die Daten gehören in eine eigene Datei oder später in eine API.

Formular lädt neu

„Anmelden“ schickt einen POST an `/anmeldung`, den es nicht gibt. Die Seite ist weg, Rückmeldung gibt es keine.

Keine Prüfung

Jede Adresse geht durch. Ob es eine HTWG-Adresse ist, merkt erst jemand, der die Anmeldungen liest.

Heute bauen wir **Stufe 4**: dasselbe HTML und CSS wie in VL 2, dazu eine Datei `app.js`. Am Ende kommt das Programm aus `programm.json`, und das Formular prüft sich selbst.

Eine Zeile im head bindet das Skript ein

```
<head>
  <link rel="stylesheet" href="styles.css">
  <script type="module" src="app.js"></script>
</head>
```

- `type="module"` wartet, bis das HTML fertig ist. Das Skript findet also alle Elemente, auch wenn es im `head` steht.
- Module erlauben später `import` und `export`, das braucht React in VL 5.
- **Konsole** öffnen mit `F12`: Dort seht ihr Fehler, und dort könnt ihr JavaScript direkt ausprobieren.

1 · Die Sprache für Java-Kenner

Was vertraut aussieht, und was euch überraschen wird

Java und JavaScript nebeneinander: vertraut, aber anders

Java

```java
int alter = 21;
alter = "einundzwanzig";   // Compilerfehler
List<String> titel = punkte.stream()
    .map(p -> p.titel()).toList();
if (a.equals(b)) { … }
```

JavaScript

```javascript
let alter = 21;
alter = "einundzwanzig";   // erlaubt!
const titel = punkte
    .map((p) => p.titel);
if (a === b) { … }
```

Typen am Wert

Kein Compiler prüft vorher. Das holt erst TypeScript zurück.

Immer ===

`"5" == 5` ist `true`, `"5" === 5` ist `false`.

const und let

Nie `var`. Funktionen sind Werte wie Zahlen.

Objekte, Arrays und Arrow Functions tragen fast jeden Code

```
const punkt = { zeit: "09:00", titel: "Kickoff" };
const { zeit, titel } = punkt;          // Destructuring

const titel2 = programm.map((p) => p.titel);
const frueh = programm.filter((p) => p.zeit < "12:00");

console.log(`${titel} beginnt um ${zeit}`);
```

Arrow Function

`(p) => p.titel` ist eine kurze Funktion, perfekt für `map` und `filter`.

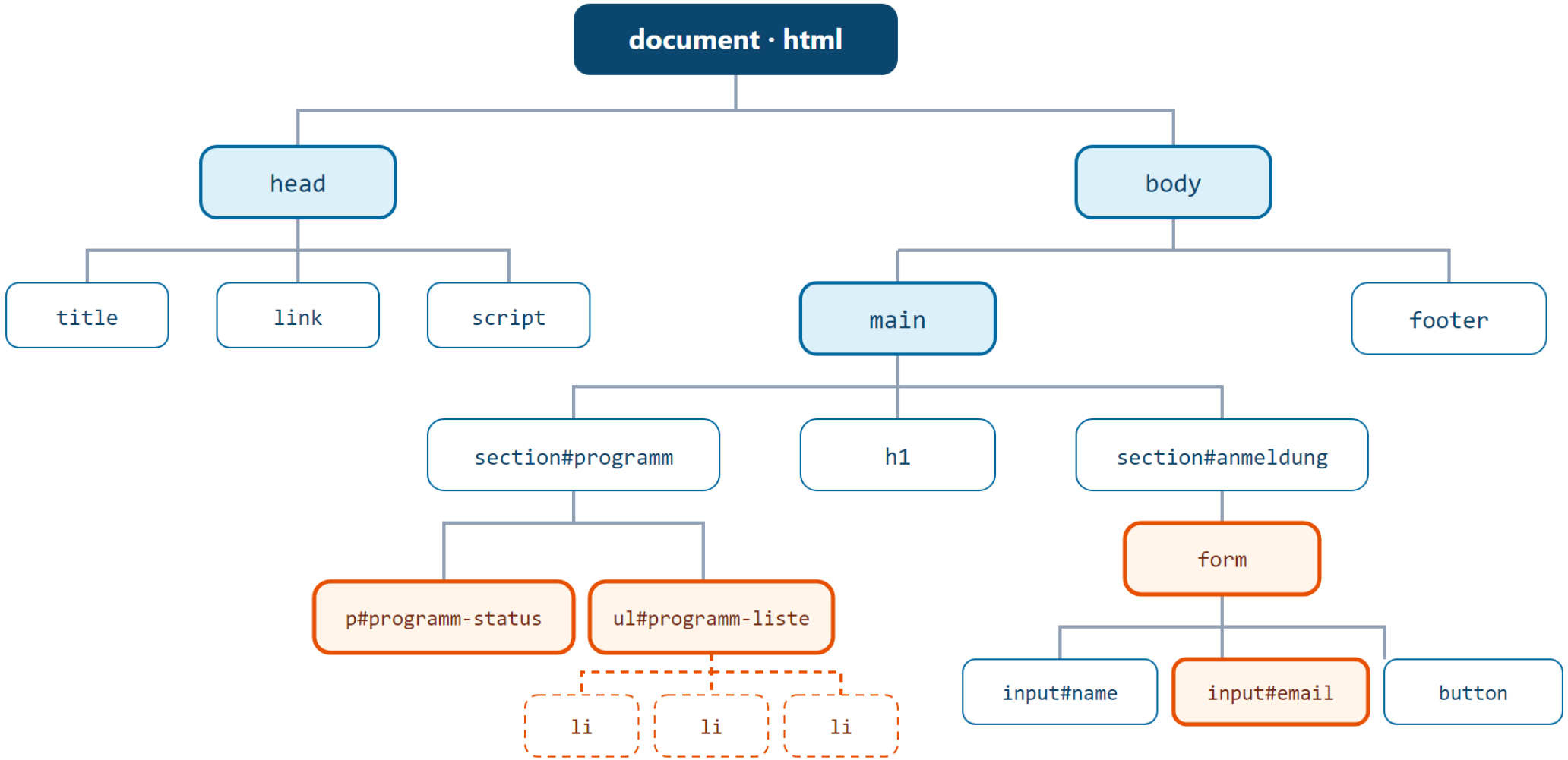
Template Literal

Backticks und `${…}` statt Strings mit `+` zusammenzukleben.

2 · Das DOM

Wie JavaScript die Seite sieht und verändert

Aus dem HTML baut der Browser einen Baum, und JavaScript arbeitet am Baum



holt app.js mit `querySelector`

erzeugt app.js aus `programm.json`

(gekürzt, Reihenfolge vereinfacht)

Live: Jede Zeile ändert sofort die Seite

Klickt eine Zeile: Sie läuft sofort gegen die Seite rechts

```
1 const karte = document.querySelector(".card");
2 karte.querySelector("h4").textContent = "09:30 · Kickoff";
3 karte.classList.add("hervorgehoben");
4 const neu = document.createElement("article");
5 neu.textContent = "19:00 · Party"; liste.append(neu);
6 p.textContent = "<b>fett?</b>";
7 p.innerHTML = "<b>fett?</b>";
```

Zurücksetzen

Kein Neuladen: Nur der Baum ändert sich, der Browser zeichnet neu.

HACKATHON-SEITE

Programm

09:30 · Kickoff
Teams finden

10:00 · Hacken
fett?

19:00 · Party

Neue Elemente baut man, statt HTML-Strings zu kleben

```
function karte({ zeit, titel, text }) {
  const article = document.createElement("article");
  article.className = "card";
  const h3 = document.createElement("h3");
  h3.textContent = `${zeit} · ${titel}`;
  const p = document.createElement("p");
  p.textContent = text;
  article.append(h3, p);
  return article;
}
```

Warum nicht `innerHTML`?

`li.innerHTML = `<h3>${titel}</h3>`` ist kürzer. Steht in den Daten aber `<img src=x onerror=…>`, führt der Browser fremden Code aus.

Das heißt **XSS**, Thema in VL 12. Mit `textContent` bleibt es Text.

3 · Events

Code, der auf Klicks und Tastendrücke wartet

addEventListener verbindet ein Ereignis mit einer Funktion

```
const email = document.querySelector("#email");

email.addEventListener("input", () => {
  const ok = email.value.trim().endsWith("@htwg-konstanz.de");
  email.classList.toggle("ungueltig", !ok);
});
```

click

Button, Link, Karte

input

bei jedem Tastendruck im Feld, ideal für Live-Prüfung

submit

auf dem `form`, nicht auf dem Button

Die Funktion wird übergeben, nicht aufgerufen. Sie läuft erst, wenn das Ereignis eintritt. Das sind die „Funktionen als Werte“ von vorhin.

preventDefault hält den Browser vom Neuladen ab

```
form.addEventListener("submit", (event) => {
  event.preventDefault();      // kein POST, kein Neuladen
  if (!pruefeEmail()) return;
  const danke = document.createElement("p");
  danke.textContent = `Danke, ${form.elements.name.value}!`;
  form.replaceWith(danke);
});
```

Ohne `preventDefault` schickt der Browser das Formular wie in VL 3 als `POST /anmeldung` ab und lädt eine neue Seite. Mit `preventDefault` übernimmt euer Code. Genau so arbeiten Single-Page-Apps wie React.

Live: Ereignisse sichtbar machen

Jedes Ereignis landet rechts im Protokoll

ANMELDUNG

Name

Lea

E-Mail

lea@gmail

Anmelden

☑ **event.preventDefault() im submit-Listener**

Tippt eine Adresse, klickt herum, schickt ab. Dann den Haken entfernen und noch einmal abschicken.

```
// Ereignis-Protokoll
focus #ev-email
input #ev-email = "l"
input #ev-email = "le"
input #ev-email = "lea"
input #ev-email = "lea@"
input #ev-email = "lea@g"
input #ev-email = "lea@gm"
input #ev-email = "lea@gma"
input #ev-email = "lea@gmai"
input #ev-email = "lea@gmail"
focus #ev-name
input #ev-name = "Lea"
submit #ev-form
→ preventDefault: Seite bleibt, Code übernimmt
```

Halbzeit

Anwesenheit per QR-Code · Testat-Häppchen

4 · Asynchron: Event Loop und Fetch

Warum die Seite nicht einfriert, während Daten laden

Rätsel: In welcher Reihenfolge erscheinen die Ausgaben?

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

JavaScript im Browser hat **einen einzigen Thread**. Trotzdem lädt die Seite Daten, während ihr scrollt. Wie geht das? Tipp: Der Timer hat 0 Millisekunden.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");

setTimeout(() => console.log("Timer"), 0);

Promise.resolve().then(() => console.log("Promise"));

console.log("Ende");
```

Konsole

Call Stack

leer

Web APIs (Browser)

leer

Microtask Queue

leer

Task Queue

leer

1/8 Das Skript startet. Alles läuft auf einem einzigen Thread.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
```

Call Stack

`console.log("Start")`

Web APIs (Browser)

leer

Microtask Queue

leer

Task Queue

leer

2/8 Zeile 1 läuft sofort auf dem Call Stack.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
```

Call Stack

`setTimeout(…, 0)`

Web APIs (Browser)

`Timer 0 ms`

Microtask Queue

leer

Task Queue

leer

3/8 setTimeout übergibt den Timer an den Browser und kehrt sofort zurück.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
```

Call Stack

leer

Web APIs (Browser)

leer

Microtask Queue

leer

Task Queue

```
() => log("Timer")
```

4/8 Der Timer ist abgelaufen. Der Callback wartet in der Task Queue.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
```

Call Stack

`Promise.then(…)`

Web APIs (Browser)

leer

Microtask Queue

`() => log("Promise")`

Task Queue

`() => log("Timer")`

5/8 Das Promise ist schon erfüllt. Der Callback wartet als Microtask.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
> Ende
```

Call Stack

`console.log("Ende")`

Web APIs (Browser)

leer

Microtask Queue

`() => log("Promise")`

Task Queue

`() => log("Timer")`

6/8 Zeile 4 läuft noch synchron, die Warteschlangen müssen warten.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
> Ende
> Promise
```

Call Stack

`log("Promise")`

Web APIs (Browser)

leer

Microtask Queue

leer

Task Queue

`() => log("Timer")`

7/8 Stack leer: Zuerst kommen alle Microtasks dran.

Der Event Loop: In welcher Reihenfolge erscheinen die Ausgaben?

app.js

```
console.log("Start");
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("Ende");
```

Konsole

```
> Start
> Ende
> Promise
> Timer
```

Call Stack

`log("Timer")`

Web APIs (Browser)

leer

Microtask Queue

leer

Task Queue

leer

8/8 Danach nimmt der Event Loop den nächsten Task. Ein Timer mit 0 ms kommt also zuletzt.

Live: Rechnen friert die Seite ein, await nicht

JavaScript hat einen Thread. Was passiert mit der Seite?

A
```
while (Date.now() < ende) {} // 3 s rechnen
```

B
```
await new Promise(r => setTimeout(r, 3000));
```

Die Uhr rechts wird von JavaScript angetrieben, 60-mal pro Sekunde. Probiert auch, während der 3 Sekunden ins Feld zu tippen.

Hier tippen …

SEITE

7.5 s

fertig: die Seite lief die ganze Zeit

async und await lesen sich wie normaler Code

```
async function ladeProgramm() {
  try {
    const antwort = await fetch("programm.json");
    if (!antwort.ok) throw new Error(`HTTP ${antwort.status}`);
    const punkte = await antwort.json();
    punkte.forEach((p) => liste.append(karte(p)));
  } catch (fehler) {
    status.textContent = `Fehler: ${fehler.message}`;
  }
}
```

await

Gibt den Thread frei, bis die Antwort da ist. Der Rest der Seite bleibt bedienbar.

⚠ fetch wirft bei 404 nicht

Nur bei Netzwerkfehlern. Den Statuscode aus VL 3 prüft ihr selbst mit `antwort.ok`.

Doppelklick auf index.html geht ab heute nicht mehr

🚫 Was passiert

Die Seite bleibt bei „Programm wird geladen …“ hängen. In der Konsole steht:

```
Access to script at 'file:///…/app.js' from origin 'null' has been blocked by CORS policy
```

✅ Was hilft

Die Seite über HTTP ausliefern, wie in VL 3:

```
python -m http.server 8000
```

oder `npx serve`

dann `http://localhost:8000`

Module und `fetch` laufen nur über HTTP. Eine Datei hat keinen Ursprung (*origin*), also blockiert der Browser. Was CORS genau ist, kommt in VL 12.

Ausblick: Module und TypeScript bringen Ordnung in große Projekte

Module

`export function karte() {…}` in einer Datei, `import { karte } from "./karte.js"` in der anderen. Jede Datei hat ihren eigenen Gültigkeitsbereich.

TypeScript

JavaScript mit Typen, die vor dem Ausführen geprüft werden:
`function karte(p: Programmpunkt): HTMLElement`
Tippfehler wie `p.titl` fallen sofort auf.

Nächste Woche bauen wir mit **Vite** ein React-Projekt mit TypeScript. Alles von heute bleibt gültig: React erzeugt DOM-Elemente, reagiert auf Events und lädt Daten mit `fetch`.

Jetzt ihr: 20 Minuten JS-Labor

Öffnet in Moodle „**VL 04 · JS-Labor**“ und ladet den Starter herunter.
Programm laden und Formular prüfen, zu zweit an einem Laptop.

Übung ab 09:45: Bringt Leben in euer eigenes Projekt

Server statt Doppelklick

Euer M1-Prototyp läuft über `python -m http.server` oder `npx serve`

Eine Interaktion

Ein Formular prüfen, eine Liste aus einer JSON-Datei laden oder etwas ein- und ausblenden

Konsole sauber

Keine roten Fehler, kein `404` im Network-Tab

Ich gehe herum und schaue mir eure M1-Abgaben mit euch an. Was ihr heute mit JavaScript baut, übernehmt ihr in M2 (React), fällig am Mittwoch, 18.11.

Quellen

- Marijn Haverbeke: *Eloquent JavaScript*. 4. Auflage, 2024, Kapitel 1 bis 4, 11, 14, 15 und 18, frei unter eloquentjavascript.net
- MDN Web Docs: *JavaScript Guide*, *Introduction to the DOM*, *Introduction to events*, *Using the Fetch API*, *Using promises*
- MDN Web Docs: *The event loop* und *In depth: Microtasks and the JavaScript runtime environment*
- TypeScript Handbook: *TypeScript for JavaScript Programmers*, typescriptlang.org

HTML beschreibt, CSS gestaltet, JavaScript reagiert.

Nächste Woche: React kompakt. Komponenten, State, Hooks und Router mit Vite.